

FLUTTER & DART

La Révolution du Développement Cross-Platform

Induction & Configuration

ENSPY - Club GI



Club - GI 2025/2026

Mission du Jour

Transformer votre machine en une **station de développement Flutter** professionnelle et créer votre premier chef-d'œuvre mobile !

2011 : Quand Google Rêvait d'un Meilleur Web I

Le Problème JavaScript

Développer Gmail ou Google Docs avec un langage qui n'a jamais été conçu pour ça. JavaScript devenait un frein pour Google.

Les Frustrations Quotidiennes :

- **Performances** Applications lourdes qui ralentissent
- **Debugging** Cauchemars de débogage sans typage
- **Scalabilité** Codebases impossibles à maintenir

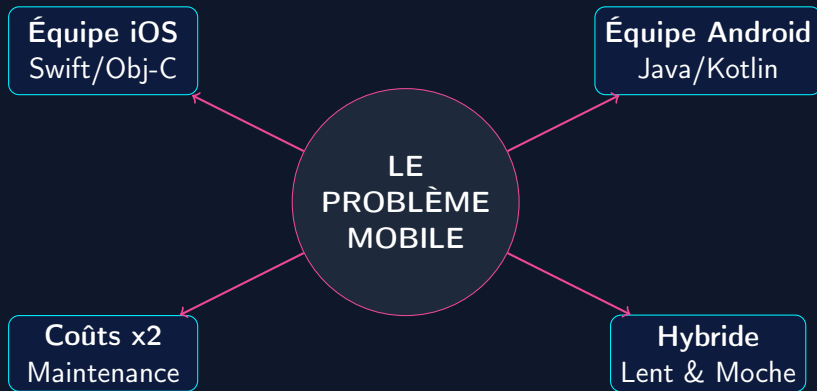
La Naissance de Dart

Google décide alors de créer son propre langage : **un JavaScript sous stéroïdes** avec tout ce qui manquait.

DART

Typage Fort
Performance Native
Outils Modernes
Syntaxe Claire

2015 : Le Mobile Était un Champ de Bataille I



2015 : Le Mobile Était un Champ de Bataille II

La Réalité du Développement Mobile Pré-Flutter

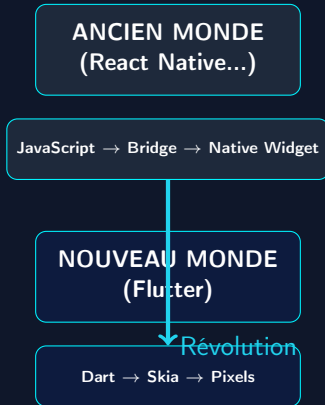
Chaque feature devait être développée deux fois, testée deux fois, maintenue deux fois. Les solutions hybrides existantes (Cordova, Ionic) offraient des performances médiocres qui ne satisfaisaient ni les développeurs ni les utilisateurs.

Flutter : La Vision Audacieuse I

Le Pari Fou de Google

Et si on arrêta d'utiliser les composants natifs d'iOS et Android ? Et si on **dessinait nous-mêmes chaque pixel** à l'écran ?

Flutter : La Vision Audacieuse II



Flutter : La Vision Audacieuse III

Résultat ? Des performances identiques au natif, une UI parfaitement identique sur toutes les plateformes, et des animations à 60-120 FPS !

La Chronologie d'une Révolution

Projet Sky
120 FPS !

Alpha
Premiers tests

Flutter 1.0
Version Stable

Flutter 2.0
Web + Desktop

AUJOURD'HUI
Version 3.x

2015

2017

2018

2021

2025

Entreprises Utilisant Flutter :

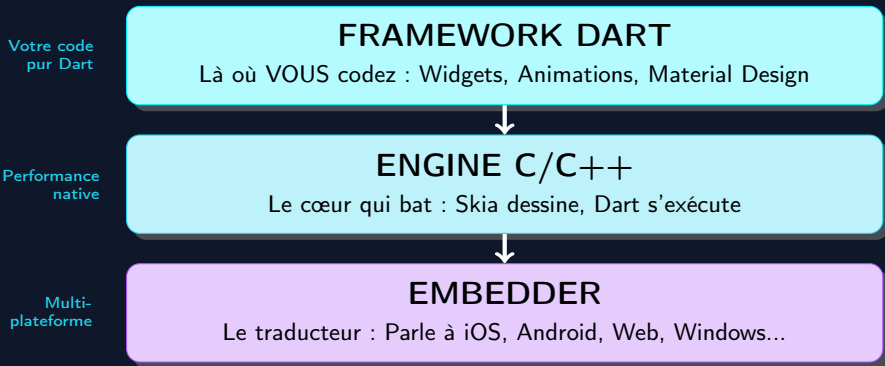
- ▶ Google (Google Pay, Ads)
- ▶ Alibaba (E-commerce)
- ▶ BMW (Applications auto)
- ▶ eBay (Marketplace)
- ▶ Nubank (Fintech)
- ▶ Toyota (Connected cars)

ARCHITECTURE

Comprendre les fondations de Flutter et son moteur



Flutter : Une Fusée à Trois Étages I



Flutter : Une Fusée à Trois Étages II

Le Principe Fondamental

Chaque couche fait exactement ce qu'elle fait le mieux. Vous ne touchez jamais au C++, et le moteur ne se soucie pas de votre UI. **Séparation parfaite des responsabilités.**

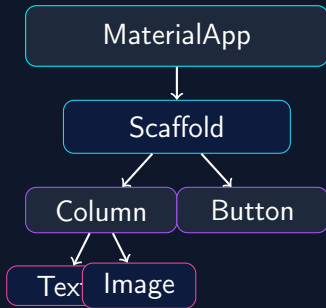
Widgets : Tout est LEGO ! !

La Philosophie Widget

Dans Flutter, ABSOLUMENT TOUT est un widget. Même les choses invisibles comme le padding ou l'alignement sont des widgets !

Exemples Concrets :

- Un bouton ? Widget.
- Du texte ? Widget.
- Un espace vide ? Widget.
- Une animation ? Widget.
- Toute votre app ? Widget.



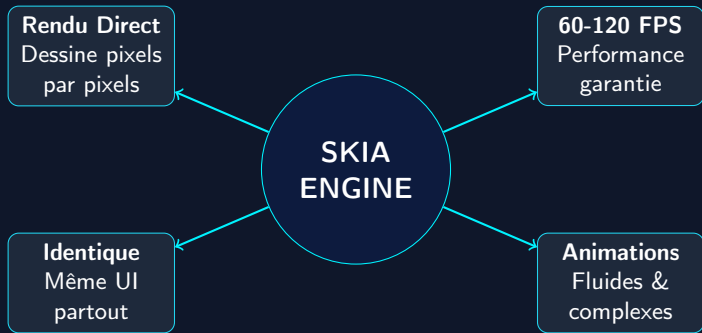
Votre app est un arbre de widgets qui s'emboîtent

Skia : Le Pinceau Magique I

Skia Graphics Engine

Skia est le moteur graphique 2D open-source qui rend Flutter unique. C'est le même moteur utilisé par Chrome, Android et Firefox. **Prouvé et testé par des milliards d'utilisateurs.**

Skia : Le Pinceau Magique II



Le Secret des Performances : JIT vs AOT

Mode Développement : JIT

Just-In-Time Compilation

Votre code Dart est compilé à la volée pendant que vous développez.

Le Superpouvoir : **HOT RELOAD**

Modifiez votre code, appuyez sur `r`, et BOOM ! Votre app se met à jour en moins d'une seconde sans perdre l'état.

Mode Production : AOT

Ahead-Of-Time Compilation

Votre code Dart est compilé en code machine natif avant même le lancement.

Le Résultat : Performances **IDENTIQUES** au natif iOS/Android

Temps de démarrage ultra-rapide, aucune compilation à l'exécution.

Développement rapide + Performance native = La combinaison parfaite

INSTALLATION

Préparer son environnement pour coder sans limite



Ce Dont Vous Avez Besoin

Processeur : Intel i5 ou équivalent (64-bit obligatoire) **RAM** : 8GB minimum, 16GB pour être confortable **Disque** : 40GB d'espace libre (avec Android SDK) **Internet** : Connexion stable pour les téléchargements

Installation : Le Rituel Magique

Les Trois Commandements

1. Télécharger le SDK Flutter depuis flutter.dev
2. L'extraire dans un dossier SANS ESPACES dans le chemin
3. L'ajouter au PATH de votre système

Windows :

```
C:\src\flutter
```

Ajoutez C:\src\flutter\bin au PATH système

Mac/Linux :

```
~/development/flutter
```

```
export PATH="$PATH:~/  
development/flutter/bin"
```

Flutter Doctor : Votre Médecin Personnel

La Commande Magique

```
flutter doctor -v
```

Cette commande analyse votre système et vous dit **exactement** ce qui fonctionne et ce qui manque. C'est votre diagnostic médical pour Flutter !

Tout est parfait
Composant OK

Attention
Fonctionnalité limitée

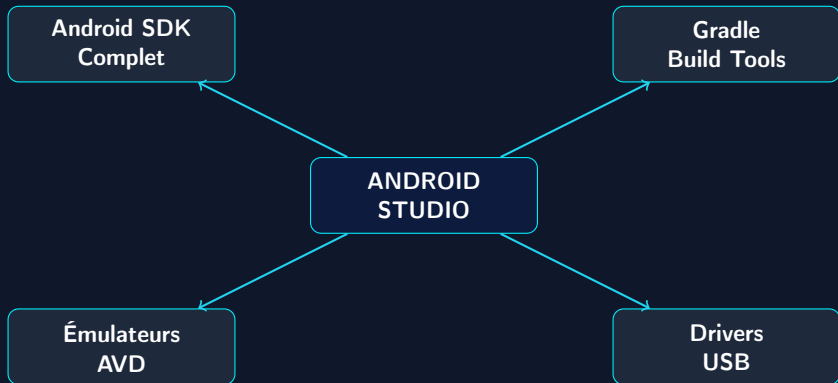
Problème
Composant manquant

Android Studio : Le Couteau Suisse I

Pourquoi Android Studio est Indispensable

Même si vous comptez coder dans VS Code ou IntelliJ, Android Studio apporte des composants **critiques** que vous ne trouverez nulle part ailleurs.

Android Studio : Le Couteau Suisse II



Android Studio : Le Couteau Suisse III

Installation : Téléchargez depuis developer.android.com/studio, lancez l'installateur, choisissez l'installation Standard, puis installez les plug-ins Flutter et Dart depuis le Marketplace.

Créer Votre Premier Émulateur I

AVD : Android Virtual Device

Un émulateur est un **smartphone virtuel** qui tourne sur votre ordinateur. Vous pouvez le configurer exactement comme un vrai téléphone !

Configuration Recommandée :

- ▶ Appareil : Pixel 6 ou 7
- ▶ Android : Version 13+
- ▶ RAM : 2048 MB minimum
- ▶ Accélération : Activée (HAXM/AMD)



Votre AVD

Créer Votre Premier Émulateur II

Astuce Pro

Créez plusieurs émulateurs : un téléphone, une tablette, et un appareil avec Android 10 pour tester la compatibilité !

Pour Ce Cours et Vos Projets

Utilisez **TOUJOURS** le canal stable. C'est la version qui a été testée par des millions de développeurs et qui ne vous laissera pas tomber en plein projet.

VOTRE PREMIERE APPLICATION

Structure de base d'un projet **Flutter**



La Commande qui Change Tout I

Créer Votre Première App Flutter

Une seule commande, et Flutter génère un projet complet avec tout ce qu'il faut :

```
flutter create mon_app_geniale  
cd mon_app_geniale  
flutter run
```

La Commande qui Change Tout II

Ce Qui Se Passe Ensuite

Flutter va :

1. Créer toute la structure de dossiers
2. Générer les fichiers de configuration pour iOS, Android, Web, etc.
3. Installer les dépendances de base
4. Lancer votre app sur l'émulateur

En moins de 2 minutes, vous avez une app qui tourne !

Anatomie d'un Projet Flutter

La Structure :

`lib/`

Votre code Dart vit ici

`android/`

Configuration Android

`ios/`

Configuration iOS

`web/`

Configuration Web

`pubspec.yaml`

Dépendances et assets

Module 0 - Flutter & Dart



95% de votre temps sera dans `lib/` - C'est là que la magie opère !

Le Fichier main.dart : Votre Point d'Entrée

Structure Basique

Chaque app Flutter commence par `main.dart`. C'est le point d'entrée de votre application.

Le Fichier main.dart : Votre Point d'Entrée II

```
import 'package:flutter/material.dart';

void main() {
  runApp(MonApplication());
}

class MonApplication extends StatelessWidget {
  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      title: 'Ma Super App',
      theme: ThemeData(primarySwatch: Colors.blue),
      home: MaPageAccueil(),
    );
  }
}
```


Hot Reload : Le Turbo du Développement II

Le Super-Pouvoir du Développeur

Modifiez un texte, changez une couleur, ajustez un padding... et voyez le résultat **instantanément** sans perdre l'état de votre app. Vous gagnez des heures chaque jour !

Commandes Essentielles

r = Hot reload (recharge le code) R = Hot restart (redémarre l'app) q = Quitter

OUTILS

Les bonnes armes pour bâtir des applis solides



VS Code vs Android Studio : Le Duel

Visual Studio Code

Les Plus :

- + Léger et rapide
- + Extensions puissantes
- + Consommation RAM faible
- + Interface épurée

Les Moins :

- Moins d'outils intégrés
- Configuration manuelle

Parfait pour le développement quotidien

Android Studio

Les Plus :

- + Outils complets intégrés
- + Meilleur pour Android natif
- + Refactoring puissant
- + Tout-en-un

Les Moins :

- Lourd (4GB RAM)
- Démarrage plus lent

Idéal pour les gros projets

Le Pub : Le Gestionnaire de Packages

Pub : Votre Bibliothèque Magique

pub est le gestionnaire de packages de Dart. C'est comme npm pour Node.js ou pip pour Python. Des milliers de packages prêts à l'emploi sur pub.dev !

Commandes Essentielles :

```
flutter pub get
```

Installe les dépendances

```
flutter pub add http
```

Ajoute un package

```
flutter pub upgrade
```

Packages Populaires :

- ▶ http - Requêtes réseau
- ▶ provider - State management
- ▶ sqflite - Base de données
- ▶ firebase_core - Firebase
- ▶ image_picker - Galerie/Caméra

Explorez pub.dev pour en découvrir des milliers d'autres !

DevTools : Le Tableau de Bord Ultime I

Flutter DevTools

Une suite complète d'outils de debugging et de profiling intégrée à Flutter. C'est votre cockpit de pilotage !

DevTools : Le Tableau de Bord Ultime II



DevTools : Le Tableau de Bord Ultime III

Accès Rapide

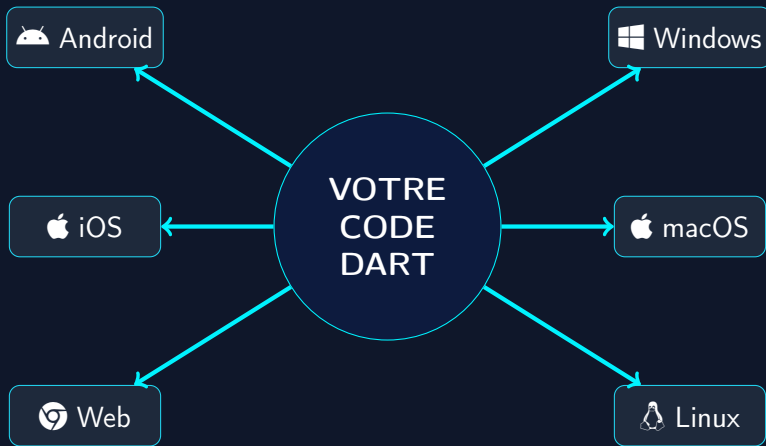
Pendant que votre app tourne, tapez simplement `flutter pub global activate devtools` puis `flutter devtools` pour ouvrir le dashboard dans votre navigateur !

MULTIPLATEFORMES

Un seul code, partout avec **Flutter**



Un Code, Six Plateformes



Material vs Cupertino : Parler Deux Langues I

Material Design

Le langage visuel de Google, utilisé sur Android et le web.

Caractéristiques :

- ◆ Flat design moderne
- ◆ Animations expressives
- ◆ Couleurs vives
- ◆ Ombres et élévations

Utilisez les widgets `Material`

Cupertino Design

Le style d'Apple, pour iOS et macOS.

Caractéristiques :

- ◆ Design minimaliste
- ◆ Transparences et flous
- ◆ Couleurs subtiles
- ◆ Navigation typique iOS

Utilisez les widgets `Cupertino`

Material vs Cupertino : Parler Deux Langues II

L'Astuce du Pro

Flutter peut automatiquement choisir le bon style selon la plateforme avec `Theme.of(context).platform!`

CONCLUSION

Flutter ouvre la voie à des applications modernes, rapides et unifiées



Merci pour votre attention

Ce Que Vous Savez Maintenant

✓ Histoire de Flutter & Dart

✓ Architecture Technique

✓ Installation Complète

✓ Première App Lancée

✓ Outils Essentiels

✓ Prêt pour le Développement

La Suite de l'Aventure

Module Suivant

Module 1 : Dart : fondations du langage

Comprendre la syntaxe et les constructions du langage **Dart** : types, null-safety, fonctions, classes, collections, asynchronisme, tests unitaires basiques.

Faites déjà des recherches pour la prochaine fois !

Vos Devoirs pour la prochaine séance

Mission Pratique

1. Vérifiez que `flutter doctor` ne montre aucune erreur
2. Créez une nouvelle app avec `flutter create`
3. Lancez-la sur l'émulateur ET en mode web
4. Modifiez le texte principal et testez le hot reload
5. Explorez pub.dev et trouvez 3 packages intéressants
6. Lisez la documentation officielle : docs.flutter.dev

Le meilleur moyen d'apprendre, c'est de PRATIQUER !

Ressources Essentielles

Documentation Officielle :

- ▶ flutter.dev
- ▶ dart.dev
- ▶ api.flutter.dev

Communauté :

- ▶ Reddit : r/FlutterDev
- ▶ Discord : Flutter Community
- ▶ StackOverflow : tag [flutter]

Packages :

- ▶ pub.dev

Tutoriels :

- ▶ Flutter Codelabs
- ▶ YouTube : Flutter Channel
- ▶ Medium : Flutter Community

Outils de Design :

- ▶ FlutterFlow (no-code)
- ▶ Figma pour Flutter

Les Erreurs à Éviter Absolument

Les Pièges du Débutant

1. Ignorer les warnings de flutter doctor

Ne passez jamais à l'étape suivante si vous avez des erreurs rouges !

2. Installer Flutter dans un dossier avec espaces

C:\Program Files\flutter × → C:\src\flutter ✓

3. Utiliser des canaux instables pour la production

Restez sur stable pour vos vrais projets !

4. Oublier de faire flutter pub get

Après chaque modification du pubspec.yaml, lancez cette commande !

5. Ne pas lire les messages d'erreur

Flutter a d'excellents messages d'erreur, lisez-les attentivement !

Tips de Pro pour Accélérer

Gagnez du Temps Dès Maintenant

1. Configurez des snippets dans votre IDE

Créez des raccourcis pour générer rapidement des StatelessWidget, StatefulWidget, etc.

2. Utilisez le widget inspector

Activez-le avec p pendant le debug mode pour explorer visuellement votre UI

3. Maîtrisez les raccourcis clavier

Hot reload (r), Hot restart (R), Toggle debug paint (p)

4. Activez le format on save

Votre code sera toujours propre et lisible automatiquement

5. Gardez DevTools ouvert

Surveillez les performances en temps réel pendant le développement

Croissance Explosive :

- ◆ +200% de téléchargements en 2023
- ◆ #1 framework mobile sur GitHub
- ◆ Supporté par Google

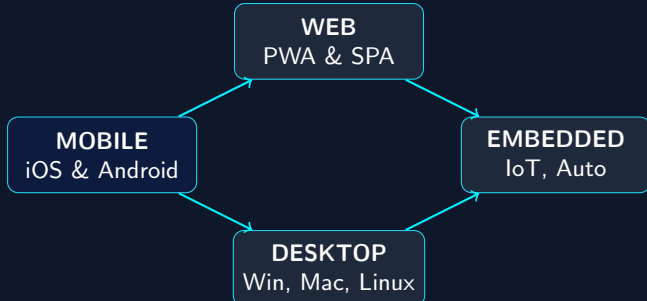
Industrie :

- ◆ E-commerce
- ◆ Fintech
- ◆ Social Media
- ◆ IoT & Automotive

L'Écosystème Flutter en Expansion

Au-delà du Mobile

Flutter n'est plus "juste" un framework mobile. C'est devenu une plateforme complète pour créer des expériences utilisateur sur TOUS les appareils.



Pourquoi les Entreprises Choisissent Flutter

Économies

Une équipe au lieu de 3+

Plus besoin d'équipes séparées pour iOS, Android et Web. Un seul développeur Flutter peut tout faire.

Time-to-market réduit

Lancez votre MVP en quelques semaines au lieu de plusieurs mois.

Qualité

UI cohérente partout

Votre marque a exactement le même look sur toutes les plateformes.

Performances natives

Les utilisateurs ne voient aucune différence avec une app native.

Votre Parcours d'Apprentissage

✓ **Niveau 1** : Installation & Configuration (AUJOURD'HUI)

→ **Niveau 2** : Widgets & Layouts

→ **Niveau 3** : State Management

→ **Niveau 4** : Navigation & Routing

→ **Niveau 5** : Backend & APIs

Mental Model pour Réussir

Changez Votre Façon de Penser

Flutter demande un changement de mentalité par rapport aux frameworks traditionnels :

Ancienne Pensée :

- ✗ "J'ai une vue à mettre à jour"
- ✗ "Je manipule le DOM"
- ✗ "J'utilise des composants natifs"
- ✗ "J'ai des templates séparés"

Pensée Flutter :

- ✓ "Je décris l'état actuel"
- ✓ "Flutter recalcule automatiquement"
- ✓ "Tout est dessiné par Skia"
- ✓ "UI = fonction de l'état"

La Formule Magique

UI = f(state)

Votre interface est toujours une fonction pure de votre état. Changez l'état, l'UI se met à jour automatiquement !

Derniers Conseils Avant de Commencer

Sagesse du Développeur Flutter

1. Soyez patient avec vous-même

Flutter a une courbe d'apprentissage, mais elle en vaut la peine !

2. Construisez tous les jours

Même 30 minutes par jour valent mieux que 5 heures le dimanche

3. Lisez le code des autres

GitHub regorge de projets Flutter open-source à étudier

4. N'ayez pas peur de casser

Expérimentez ! Le pire qui puisse arriver est un message d'erreur

5. Rejoignez la communauté

Les développeurs Flutter sont accueillants et adorent aider les débutants

QUESTIONS ?

Vos idées et interrogations sont les bienvenues



Prêt à révolutionner le développement mobile ?

LET'S BUILD!

Échangeons ensemble

//

*Le meilleur moment pour apprendre Flutter
était hier.*

*Le deuxième meilleur moment
c'est aujourd'hui.*

//

Commencez à coder. Créez. Partagez.

L'aventure Flutter ne fait que commencer.